Name:- Soham kulkarni

Class:- Sy-A

RollNo.:- A-58

Batch:- A4

Aim: Use sequence data types and their associated operations in Python a) Strings b) Lists c) Arrays d) Tuples e) Dictionary f) Sets g) Range

Assignment1:-

1.What are sequence data types in Python? Name them and explain how indexing and slicing work across different sequence types?

Ans:- Sequence data types in Python are collections of ordered elements where each item is stored in a specific position. The main sequence data types include lists, tuples, strings, and range objects. These data types allow accessing elements using indexing and slicing. Indexing means accessing a specific element using its position, where indexing starts from 0 for the first element and -1 for the last element. Slicing allows extracting a portion of the sequence by specifying a start index and an end index. This concept works similarly across all sequence types, making it easy to access and manipulate data efficiently.

2.What is the difference between a list and a tuple in Python? Why would you prefer a tuple over a list in real-world applications?

Ans:- Lists and tuples are both used to store multiple elements, but the main difference is that lists are mutable while tuples are immutable. This means that elements in a list can be changed, added, or removed, whereas elements in a tuple cannot be modified after creation. Lists use square brackets [], while tuples use parentheses (). Tuples are preferred in real-world applications when the data should not be changed, such as storing fixed data like coordinates, configuration values, or records. They are also faster and more memory-efficient compared to lists.

3.How are Python arrays different from lists? When should you use the array module instead of a list?

Ans:- Python arrays and lists are both used to store collections of data, but they differ in functionality and usage. Lists can store elements of different data types, such as integers, strings, and floats, in a single structure. In contrast, arrays (from the array module) can store only elements of the same data type, making them more memory efficient for numerical data. Arrays are typically used when dealing

with large amounts of numerical data where performance and memory optimization are important, whereas lists are more flexible and commonly used for general purposes.

4.Explain how dictionaries work internally in Python. Why are dictionary lookups faster compared to lists or tuples?

Ans:- Dictionaries in Python store data in key-value pairs and work internally using a data structure called a hash table. When a key-value pair is added, Python uses a hash function to convert the key into a unique index where the value is stored. This allows direct access to the value using its key instead of searching through all elements. Because of this hashing mechanism, dictionary lookups are much faster, typically taking constant time (O(1)), compared to lists or tuples, which require linear search (O(n)). This makes dictionaries highly efficient for searching and retrieving data.

5.What is the difference between sets and lists in Python? How do sets handle duplicate values, and where are sets useful in practical scenarios?

Ans:- Sets and lists are both used to store collections of elements, but they differ in important ways. Lists are ordered collections that allow duplicate values and support indexing. Sets, on the other hand, are unordered collections that do not allow duplicate elements. When duplicate values are added to a set, they are automatically removed, ensuring that each element is unique. Sets are useful in practical scenarios such as removing duplicates from data, performing mathematical operations like union and intersection, and checking membership quickly. Lists are more suitable when order and duplicates are important.

a)Strings

In [ ]:
```
# Original String
text = "Hello World"

print("Original String:", text)

# 1. Convert to Uppercase
print("Uppercase:", text.upper())

# 2. Convert to Lowercase
print("Lowercase:", text.lower())

# 3. Length of String
print("Length:", len(text))

# 4. Access Characters (Indexing)
print("First character:", text[0])
print("Last character:", text[-1])
```

```python
# 5. String Slicing
print("Slice (0-5):", text[0:5])
print("Slice (6-end):", text[6:])

# 6. Replace a word
print("Replace World with Python:", text.replace("World", "Python"))

# 7. Check substring
print("Is 'Hello' present?", "Hello" in text)

# 8. Count occurrences
print("Count of 'l':", text.count('l'))

# 9. Find position
print("Position of 'World':", text.find("World"))

# 10. Split string
print("Split:", text.split())

# 11. Join strings
words = ["Hello", "World"]
print("Join:", " ".join(words))

# 12. Remove spaces
text2 = "   Hello   "
print("Strip:", text2.strip())

# 13. Startswith and Endswith
print("Starts with 'Hello'?", text.startswith("Hello"))
print("Ends with 'World'?", text.endswith("World"))

# 14. Check string type
print("Is alphabet?", text.isalpha())
print("Is digit?", text.isdigit())

# 15. Reverse string
print("Reversed:", text[::-1])
```

```
Original String: Hello World
Uppercase: HELLO WORLD
Lowercase: hello world
Length: 11
First character: H
Last character: d
Slice (0-5): Hello
Slice (6-end): World
Replace World with Python: Hello Python
Is 'Hello' present? True
Count of 'l': 3
Position of 'World': 6
Split: ['Hello', 'World']
Join: Hello World
Strip: Hello
Starts with 'Hello'? True
Ends with 'World'? True
Is alphabet? False
Is digit? False
Reversed: dlroW olleH
```

b)Lists

In [ ]:

```python
# 1. Create List
my_list = [10, 20, 30, 40, 50]
print("Original List:", my_list)

# 2. Access Elements (Indexing)
print("First element:", my_list[0])
print("Last element:", my_list[-1])

# 3. List Slicing
print("Elements from index 1 to 3:", my_list[1:4])

# 4. Add Elements
my_list.append(60)   # add at end
print("After append:", my_list)

my_list.insert(2, 25)  # insert at index
print("After insert:", my_list)

# 5. Extend List
my_list.extend([70, 80])
print("After extend:", my_list)

# 6. Remove Elements
my_list.remove(25)   # remove specific value
print("After remove:", my_list)

my_list.pop()        # remove last element
print("After pop:", my_list)

# 7. Update Elements
```

```
my_list[1] = 22
print("After update:", my_list)

# 8. List Length
print("Length of list:", len(my_list))

# 9. Sorting List
my_list.sort()
print("Sorted list:", my_list)

# 10. Reverse List
my_list.reverse()
print("Reversed list:", my_list)

# 11. Copy List
copy_list = my_list.copy()
print("Copied list:", copy_list)

# 12. Count Elements
print("Count of 40:", my_list.count(40))

# 13. Find Index
print("Index of 40:", my_list.index(40))

# 14. Check Element
print("Is 30 in list?", 30 in my_list)

# 15. Clear List
temp_list = [1, 2, 3]
temp_list.clear()
print("After clear:", temp_list)
```

```
Original List: [10, 20, 30, 40, 50]
First element: 10
Last element: 50
Elements from index 1 to 3: [20, 30, 40]
After append: [10, 20, 30, 40, 50, 60]
After insert: [10, 20, 25, 30, 40, 50, 60]
After extend: [10, 20, 25, 30, 40, 50, 60, 70, 80]
After remove: [10, 20, 30, 40, 50, 60, 70, 80]
After pop: [10, 20, 30, 40, 50, 60, 70]
After update: [10, 22, 30, 40, 50, 60, 70]
Length of list: 7
Sorted list: [10, 22, 30, 40, 50, 60, 70]
Reversed list: [70, 60, 50, 40, 30, 22, 10]
Copied list: [70, 60, 50, 40, 30, 22, 10]
Count of 40: 1
Index of 40: 3
Is 30 in list? True
After clear: []
```

c)Arrays

In [ ]:
```
import numpy as np
```

```python
# 1. Create Array
arr = np.array([10, 20, 30, 40, 50])
print("Original Array:", arr)

# 2. Access Elements (Indexing)
print("First element:", arr[0])
print("Last element:", arr[-1])

# 3. Array Slicing
print("Elements from index 1 to 3:", arr[1:4])

# 4. Add Elements
arr = np.append(arr, 60)
print("After adding element:", arr)

# 5. Remove Elements
arr = np.delete(arr, 2)  # remove element at index 2
print("After deleting element:", arr)

# 6. Update Element
arr[1] = 25
print("After updating element:", arr)

# 7. Array Arithmetic Operations
arr2 = np.array([1, 2, 3, 4, 5])

print("Addition:", arr + arr2)
print("Subtraction:", arr - arr2)
print("Multiplication:", arr * arr2)
print("Division:", arr / arr2)

# 8. Sum and Mean
print("Sum:", np.sum(arr))
print("Mean:", np.mean(arr))

# 9. Maximum and Minimum
print("Maximum:", np.max(arr))
print("Minimum:", np.min(arr))

# 10. Sort Array
print("Sorted Array:", np.sort(arr))

# 11. Reshape Array
arr3 = np.array([1, 2, 3, 4, 5, 6])
print("Reshaped (2x3):")
print(arr3.reshape(2, 3))

# 12. Find Index of Element
print("Index of 40:", np.where(arr == 40))

# 13. Copy Array
copy_arr = arr.copy()
```

```
print("Copied Array:", copy_arr)

# 14. Reverse Array
print("Reversed Array:", arr[::-1])
```

```
Original Array: [10 20 30 40 50]
First element: 10
Last element: 50
Elements from index 1 to 3: [20 30 40]
After adding element: [10 20 30 40 50 60]
After deleting element: [10 20 40 50 60]
After updating element: [10 25 40 50 60]
Addition: [11 27 43 54 65]
Subtraction: [ 9 23 37 46 55]
Multiplication: [ 10  50 120 200 300]
Division: [10.         12.5        13.33333333 12.5        12.        ]
Sum: 185
Mean: 37.0
Maximum: 60
Minimum: 10
Sorted Array: [10 25 40 50 60]
Reshaped (2x3):
[[1 2 3]
 [4 5 6]]
Index of 40: (array([2]),)
Copied Array: [10 25 40 50 60]
Reversed Array: [60 50 40 25 10]
```

d)Tuples

In [ ]:
```
# 1. Create a Tuple
my_tuple = (10, 20, 30, 40, 50)
print("Original Tuple:", my_tuple)

# 2. Access Elements (Indexing)
print("First element:", my_tuple[0])
print("Last element:", my_tuple[-1])

# 3. Tuple Slicing
print("Slicing (1 to 3):", my_tuple[1:4])

# 4. Length of Tuple
print("Length:", len(my_tuple))

# 5. Count Occurrence
print("Count of 20:", my_tuple.count(20))

# 6. Find Index
print("Index of 30:", my_tuple.index(30))

# 7. Membership Check
print("Is 40 present?", 40 in my_tuple)

# 8. Loop Through Tuple
```

```python
print("Elements in tuple:")
for item in my_tuple:
    print(item)

# 9. Concatenate Tuples
tuple2 = (60, 70)
new_tuple = my_tuple + tuple2
print("After concatenation:", new_tuple)

# 10. Repeat Tuple
print("Repeated Tuple:", my_tuple * 2)

# 11. Convert Tuple to List (for modification)
temp_list = list(my_tuple)
temp_list.append(100)
new_tuple = tuple(temp_list)
print("After adding element (using list):", new_tuple)

# 12. Nested Tuple
nested_tuple = (1, (2, 3), (4, 5))
print("Nested Tuple:", nested_tuple)
print("Access nested element:", nested_tuple[1][0])

# 13. Unpacking Tuple
a, b, c, d, e = my_tuple
print("Unpacked values:", a, b, c, d, e)

# 14. Delete Tuple
temp = (1, 2, 3)
del temp
print("Tuple deleted successfully")
```

```
Original Tuple: (10, 20, 30, 40, 50)
First element: 10
Last element: 50
Slicing (1 to 3): (20, 30, 40)
Length: 5
Count of 20: 1
Index of 30: 2
Is 40 present? True
Elements in tuple:
10
20
30
40
50
After concatenation: (10, 20, 30, 40, 50, 60, 70)
Repeated Tuple: (10, 20, 30, 40, 50, 10, 20, 30, 40, 50)
After adding element (using list): (10, 20, 30, 40, 50, 100)
Nested Tuple: (1, (2, 3), (4, 5))
Access nested element: 2
Unpacked values: 10 20 30 40 50
Tuple deleted successfully
```

e)Dictionary

In [ ]:

```
# 1. Create a Dictionary
my_dict = {"name": "Gajanan", "age": 20, "branch": "Engineering"}
print("Original Dictionary:", my_dict)

# 2. Access Values
print("Name:", my_dict["name"])
print("Age:", my_dict.get("age"))

# 3. Add New Key-Value Pair
my_dict["college"] = "MIT"
print("After adding:", my_dict)

# 4. Update Value
my_dict["age"] = 21
print("After update:", my_dict)

# 5. Remove Elements
my_dict.pop("branch")   # remove specific key
print("After pop:", my_dict)

# 6. Remove Last Item
my_dict.popitem()
print("After popitem:", my_dict)

# 7. Add Multiple Values
my_dict.update({"city": "Pune", "country": "India"})
print("After update multiple:", my_dict)

# 8. Length of Dictionary
print("Length:", len(my_dict))

# 9. Keys, Values, Items
print("Keys:", my_dict.keys())
print("Values:", my_dict.values())
print("Items:", my_dict.items())

# 10. Check Key Exists
print("Is 'name' present?", "name" in my_dict)

# 11. Loop Through Dictionary
print("Loop through keys:")
for key in my_dict:
    print(key)

print("Loop through values:")
for value in my_dict.values():
    print(value)

print("Loop through key-value pairs:")
for key, value in my_dict.items():
    print(key, ":", value)
```

```python
# 12. Copy Dictionary
copy_dict = my_dict.copy()
print("Copied Dictionary:", copy_dict)

# 13. Nested Dictionary
nested_dict = {
    "student1": {"name": "A", "marks": 80},
    "student2": {"name": "B", "marks": 90}
}
print("Nested Dictionary:", nested_dict)
print("Access nested value:", nested_dict["student1"]["marks"])

# 14. Clear Dictionary
temp_dict = {"a": 1, "b": 2}
temp_dict.clear()
print("After clear:", temp_dict)

# 15. Delete Dictionary
temp = {"x": 10}
del temp
print("Dictionary deleted successfully")
```

```
Original Dictionary: {'name': 'Gajanan', 'age': 20, 'branch': 'Engineering'}
Name: Gajanan
Age: 20
After adding: {'name': 'Gajanan', 'age': 20, 'branch': 'Engineering', 'colleg
e': 'MIT'}
After update: {'name': 'Gajanan', 'age': 21, 'branch': 'Engineering', 'colleg
e': 'MIT'}
After pop: {'name': 'Gajanan', 'age': 21, 'college': 'MIT'}
After popitem: {'name': 'Gajanan', 'age': 21}
After update multiple: {'name': 'Gajanan', 'age': 21, 'city': 'Pune', 'countr
y': 'India'}
Length: 4
Keys: dict_keys(['name', 'age', 'city', 'country'])
Values: dict_values(['Gajanan', 21, 'Pune', 'India'])
Items: dict_items([('name', 'Gajanan'), ('age', 21), ('city', 'Pune'), ('countr
y', 'India')])
Is 'name' present? True
Loop through keys:
name
age
city
country
Loop through values:
Gajanan
21
Pune
India
Loop through key-value pairs:
name : Gajanan
age : 21
city : Pune
country : India
Copied Dictionary: {'name': 'Gajanan', 'age': 21, 'city': 'Pune', 'country': 'I
ndia'}
Nested Dictionary: {'student1': {'name': 'A', 'marks': 80}, 'student2': {'nam
e': 'B', 'marks': 90}}
Access nested value: 80
After clear: {}
Dictionary deleted successfully
```

f)Sets

In [ ]:
```python
# 1. Create a Set
my_set = {10, 20, 30, 40, 50}
print("Original Set:", my_set)

# 2. Add Elements
my_set.add(60)
print("After add:", my_set)

# 3. Add Multiple Elements
my_set.update([70, 80])
print("After update:", my_set)
```

```python
# 4. Remove Elements
my_set.remove(20)   # removes specific element
print("After remove:", my_set)

# 5. Discard Element (no error if not present)
my_set.discard(100)
print("After discard:", my_set)

# 6. Pop Element (random removal)
removed = my_set.pop()
print("After pop:", my_set)
print("Removed element:", removed)

# 7. Length of Set
print("Length:", len(my_set))

# 8. Membership Check
print("Is 30 present?", 30 in my_set)

# 9. Loop Through Set
print("Elements in set:")
for item in my_set:
    print(item)

# 10. Set Operations
set1 = {1, 2, 3, 4}
set2 = {3, 4, 5, 6}

print("Union:", set1.union(set2))
print("Intersection:", set1.intersection(set2))
print("Difference (set1 - set2):", set1.difference(set2))
print("Symmetric Difference:", set1.symmetric_difference(set2))

# 11. Copy Set
copy_set = my_set.copy()
print("Copied Set:", copy_set)

# 12. Clear Set
temp_set = {1, 2, 3}
temp_set.clear()
print("After clear:", temp_set)

# 13. Frozen Set (Immutable Set)
frozen = frozenset([1, 2, 3, 4])
print("Frozen Set:", frozen)

# 14. Subset and Superset
A = {1, 2}
B = {1, 2, 3, 4}

print("A is subset of B:", A.issubset(B))
print("B is superset of A:", B.issuperset(A))
```

```
Original Set: {50, 20, 40, 10, 30}
After add: {50, 20, 40, 10, 60, 30}
After update: {80, 50, 20, 70, 40, 10, 60, 30}
After remove: {80, 50, 70, 40, 10, 60, 30}
After discard: {80, 50, 70, 40, 10, 60, 30}
After pop: {50, 70, 40, 10, 60, 30}
Removed element: 80
Length: 6
Is 30 present? True
Elements in set:
50
70
40
10
60
30
Union: {1, 2, 3, 4, 5, 6}
Intersection: {3, 4}
Difference (set1 - set2): {1, 2}
Symmetric Difference: {1, 2, 5, 6}
Copied Set: {50, 70, 40, 10, 60, 30}
After clear: set()
Frozen Set: frozenset({1, 2, 3, 4})
A is subset of B: True
B is superset of A: True
```

g)Range

In [ ]:

```python
# 1. Basic Range (0 to n-1)
print("Range(5):")
for i in range(5):
    print(i)

# 2. Range with Start and Stop
print("\nRange(2, 7):")
for i in range(2, 7):
    print(i)

# 3. Range with Step
print("\nRange(1, 10, 2):")
for i in range(1, 10, 2):
    print(i)

# 4. Negative Step (Reverse)
print("\nRange(10, 0, -2):")
for i in range(10, 0, -2):
    print(i)

# 5. Convert Range to List
r = range(5)
print("\nRange as list:", list(r))

# 6. Check Membership
```

```python
print("\nIs 3 in range(5)?", 3 in range(5))
print("Is 10 in range(5)?", 10 in range(5))

# 7. Length of Range
print("\nLength of range(5):", len(range(5)))

# 8. Indexing in Range
r = range(10)
print("\nElement at index 3:", r[3])
print("Last element:", r[-1])

# 9. Slicing Range
print("\nSliced range:", list(r[2:8:2]))

# 10. Use Range in Loop (Sum of numbers)
total = 0
for i in range(1, 6):
    total += i
print("\nSum from 1 to 5:", total)
```

```
Range(5):
0
1
2
3
4

Range(2, 7):
2
3
4
5
6

Range(1, 10, 2):
1
3
5
7
9

Range(10, 0, -2):
10
8
6
4
2

Range as list: [0, 1, 2, 3, 4]

Is 3 in range(5)? True
Is 10 in range(5)? False

Length of range(5): 5

Element at index 3: 3
Last element: 9

Sliced range: [2, 4, 6]

Sum from 1 to 5: 15
```

Conclusion:

This practical helped in understanding the use of different sequence data types in Python such as strings, lists, arrays, tuples, dictionaries, sets, and range. Through various operations, we learned how to store, access, and manipulate data efficiently using indexing, slicing, and built-in functions. Each data type has its own characteristics and advantages, making them suitable for different applications. Overall, this practical improved our knowledge of handling data structures effectively in Python programming.